Figure 1: Twitter keyword search area

```
# read tweets word

read_tweets = searchTwitter("womens+reservation", n=1500, lan
g="en",geocode='21.14,79.08,1000mi')
```

The + sign within the search string indicates a search for both the strings.

```
# extract the text from tweets
tweeter_txt = sapply(read_tweets, function(x) x$getText())
```

Since Twitter data often contains special characters along with different graphical icons, it is required to filter out all non-alphabetic content from the tweets. This filtering has been done by removing retweets, @, punctuations, numbers, HTML links, extra white spaces and all the crazy characters. To make all the text uniform, globally, everything has been converted to lower case letters. Finally, all the NAs have been removed and the header tag names made empty with NULLs.

```
# remove at people
twitter_txt = gsub("@\\w+", "", twitter_txt)
#"crazy" characters, just convert them to ASCII
twitter_txt = iconv(twitter_txt, to = "ASCII//TRANSLIT")
#Create a corpus from the twitter data
docsCorpus <- Corpus(VectorSource(twitter_txt))
#remove all Punctuation
docsNoPunc<- tm_map(docsCorpus, removePunctuation)
#Remove all numbers
docsNoNum<- tm_map(docsNoPunc, removeNumbers)
# Convert all to lower case
docsLower <- tm_map(docsNoNum, tolower)
# remove stop words
docsNoWords<- tm_map(docsLower, removeWords,
stopwords("english"))
#remove space
docsNoSpace<- tm_map(docsNoWords, stripWhitespace)
# Convert corpus to text
docsList = lapply(docsNoSpace,as.character)
twitter_txt = as.character(docsList)
# Replace all attributes to NULL
names(twitter_txt) = NULL
```

The next task is to classify the Twitter corpus into different classes. For this, I have used the Bayesian algorithm with *prior* equal to 1. In this case, the emotion classifier will try to evaluate each sentence with respect to its six sentiment measures, and will finally assign a classification measure to each sentence. All the unclassified sentences are categorised as NA and are finally marked as 'unknown'.

```
# classify emotion
class_emo = classify emotion(twitter_txt, algorithm="bayes",
prior=1.0)
# get emotion best fit

>head(class_emo,1)
     ANGER                    DISGUST              FEAR
[1,] "1.46871776464786" "3.09234031207392" "2.06783599555953"
     JOY                      SADNESS              SURPRISE
BEST_FIT
[1,] "1.02547755260094" "1.7277074477352" "2.78695866252273"
NA

emotion = class_emo[,7]

# substitute NA's by "unknown"

emotion[is.na(emotion)] = "unknown"
```

Classification of sentences requires the polarity measure of each sentence on the basis of its sentiment values. There are three sentiment measures — neutral, positive and negative. The polarity measure function *classify_pol()* assigns one of these three sentiment measures to each sentence. This function uses the Bayesian algorithm to calculate the final sentiment polarities of each sentence of the corpus.

```
# classify polarity
class_pol = classify_polarity(twitter_txt, algorithm="bayes")

>head(class_pol,1)
       POS                NEG                      POS/NEG
BEST_FIT
[1,] "24.2844130953411" "18.5054868578024" "1.3122817725329"
"neutral"
```